

SmartHR

Employee & HR Management System

FULL-STACK WEB APPLICATION

Technology Stack: React · Node.js · Express · MongoDB · Mongoose · JWT

Team Members	[Team member names]
--------------	---------------------

Team Lead	[Team lead name]
-----------	------------------

Supervisor / Instructor	[Supervisor / instructor name]
-------------------------	--------------------------------

Organization / Training Program	[Organization or training program name]
---------------------------------	---

Date	[Submission date]
------	-------------------

Table of Contents

1. Project Overview

5

2. Project Objectives

5

2.1 Centralize HR Data

6

2.2 Improve HR Operations

6

2.3 Role-Based Access

6

2.4 Improve Employee Experience

6

2.5 Support Management

6

2.6 Improve Security

6

2.7 Provide Scalable Architecture

6

3. Project Scope

6

4. System Users & Roles

8

4.1 Permissions Matrix

10

5. Functional Requirements

11

5.1 Authentication

11

5.2 User Management

11

5.3 Employee Management

11

5.4 Departments

12

5.5 Positions

12

5.6 Attendance

12

5.7 Leaves

13

5.8 Tasks

13

5.9 Payroll

14

5.10 Documents	14
5.11 Evaluations	14
5.12 Notifications	15
5.13 Dashboards	15
5.14 Profile & Settings	15
6. Non-Functional Requirements	16
7. System Architecture	16
8. Technology Stack	18
9. Database Overview	19
10. Main Relationships	20
11. Authentication & Authorization Flow	21
12. Core System Workflows	22
12.1 Employee Creation Workflow	22
12.2 Leave Request Workflow	22
12.3 Task Workflow	23
12.4 Attendance Workflow	23
12.5 Payroll Workflow	24
12.6 Evaluation Workflow	24
13. Frontend & Backend Responsibilities	24
14. API Communication Overview	25
15. UI/UX Design Principles	26

16. Main Application Screens	27
17. Error Handling & System States	27
18. Security Overview	28
19. File Management	28
20. Project Structure Overview	29
21. Development & Collaboration	29
22. Testing Overview	30
23. Future Enhancements	30
24. Documentation Structure	31
25. Glossary	33
26. Conclusion	33

1. Project Overview

SmarHR is a web-based Employee & HR Management System designed to centralize and simplify common Human Resources operations in a single platform. Instead of relying on scattered spreadsheets, paper records, or disconnected tools, SmarHR brings employee information and HR workflows together into one consistent system.

The system provides different functionalities for different user roles, allowing each type of user to work with the information and tools relevant to their responsibilities. The main functional areas of SmarHR include:

- Employee management
- User and authentication management
- Department management
- Position management
- Attendance management
- Leave management
- Task management
- Payroll management
- Document management
- Employee evaluations
- Notifications
- Dashboards and reports

Main idea. SmarHR connects employees, HR staff, managers, accountants, and system administrators through one centralized platform, so that each participant in the HR process has a clear, role-appropriate view of the information they need.

Problem addressed. Traditional HR processes often depend on manual records, spreadsheets, disconnected systems, and back-and-forth communication between departments. This creates inconsistent data, slow processes, and limited visibility into the overall state of the organization's HR activities.

SmarHR aims to provide:

- Centralized employee information
- Role-based access to system features
- Easier and more consistent HR operations
- Better organization of HR-related data
- Improved communication between employees, managers, and HR
- Easier tracking of employee activities
- Better visibility into overall HR processes

2. Project Objectives

This section outlines the main objectives that guided the design of SmarHR.

2.1 Centralize HR Data

Store employee and HR-related information in one centralized system, so that data about employees, departments, attendance, leaves, payroll, documents, and evaluations is no longer scattered across separate tools.

2.2 Improve HR Operations

Simplify common HR activities such as attendance tracking, leave requests, payroll processing, document handling, and performance evaluations, reducing manual effort and the chance of error.

2.3 Role-Based Access

Ensure that each user can access only the features and information appropriate to their role, protecting sensitive HR and financial data while still giving every role the tools they need.

2.4 Improve Employee Experience

Allow employees to manage and view their own information, attendance, leaves, tasks, payroll, documents, and evaluations directly through the system, reducing dependence on manual requests to HR.

2.5 Support Management

Provide managers with tools to manage their teams, assign tasks, review attendance, approve or reject leave requests, and evaluate employees under their supervision.

2.6 Improve Security

Protect user accounts and system resources through authentication, authorization, password security, and controlled access to sensitive operations and data.

2.7 Provide Scalable Architecture

Design the system so that additional HR features can be added in the future without completely redesigning the application, supporting long-term growth of the platform.

3. Project Scope

This section clearly defines what is included in the SmarHR project. The scope is organized into functional areas, each representing a group of related features.

Authentication & Authorization

- Login
- Password management
- JWT-based authentication
- Role-based authorization
- Protected resources

User Management

- User accounts
- Roles
- Account status

- User administration

Employee Management

- Employee profiles
- Employee codes
- Departments
- Positions
- Managers
- Employment information
- Salary information
- Employee status

Organization Management

- Departments
- Positions
- Department managers
- Position information

Attendance

- Check-in
- Check-out
- Attendance records
- Attendance status
- Late records
- Remote attendance
- Working hours

Leave Management

- Leave requests
- Leave types
- Leave dates
- Approval / rejection
- Rejection reasons
- Leave history

Task Management

- Create tasks
- Assign tasks
- Task priorities
- Task status
- Due dates
- Task tracking

Payroll

- Salary information
- Allowances
- Bonuses
- Deductions
- Overtime pay
- Tax
- Net salary
- Payroll status

Documents

- Employee documents
- Document upload
- Document metadata
- Document preview
- Document management

Evaluations

- Employee evaluations
- Evaluator information
- Performance scores
- Comments
- Overall score

Notifications

- System notifications
- Leave notifications
- Task notifications
- Payroll notifications
- Document notifications
- Evaluation notifications

4. System Users & Roles

SmartHR defines five system roles, each with a distinct area of responsibility. Role-based authorization is enforced by the backend, while the frontend also hides or restricts UI features based on the permissions of the currently authenticated user.

SUPER_ADMIN

Has full access to the system.

Main access:

- Manage users

- Manage employees
- Manage departments
- Manage positions
- Manage attendance
- Manage leaves
- Manage tasks
- Manage payroll
- Manage documents
- Manage evaluations
- Manage notifications
- Manage system-level settings

HR

Responsible for HR operations.

Main access:

- Employees
- Departments
- Positions
- Attendance
- Leaves
- Documents
- User-related HR operations
- Employee evaluations where applicable

MANAGER

Responsible for managing their team.

Main access:

- Team employees
- Team attendance
- Team leaves
- Tasks
- Team evaluations

ACCOUNTANT

Responsible mainly for payroll and financial employee information.

Main access:

- Payroll
- Payroll records
- Employee basic information required for payroll

- Payroll reports

EMPLOYEE

Can access their own information and employee-related services.

Main access:

- My Profile
- My Attendance
- My Leaves
- My Tasks
- My Payroll
- My Documents
- My Evaluations
- Notifications

In addition to their access areas, Managers can assign tasks, review team attendance, approve or reject leave requests, and evaluate employees on their team.

4.1 Permissions Matrix

The table below presents a conceptual permissions matrix showing which modules each role can access. A checkmark (✓) indicates that the role has access to that module; a dash (—) indicates the module is outside that role's normal responsibilities.

	SUPER_ADMIN	HR	MANAGER	ACCOUNTANT	EMPLOYEE (own data)
Users	✓	✓	✓	✓	✓
Employees	✓	✓	✓	✓	✓
Departments & Positions	✓	✓	—	—	—
Attendance	✓	✓	✓	—	✓
Leaves	✓	✓	✓	—	✓
Tasks	✓	—	✓	—	✓
Payroll	✓	—	—	✓	✓
Documents	✓	✓	—	—	✓
Evaluations	✓	✓	✓	—	✓
Notifications	✓	✓	✓	✓	✓
System Settings	✓	—	—	—	—

Important: role-based authorization is enforced by the backend on every protected endpoint. The frontend also hides or restricts UI features based on permissions, which improves the user experience but is not, by itself, a security boundary.

5. Functional Requirements

This section defines the functional requirements of SmartHR, organized by module. Each requirement is identified with an ID (FR-xx) and describes expected system behavior at a conceptual level, without prescribing implementation details.

5.1 Authentication

Purpose: Verify user identity and issue an authenticated session.

Main users: All users

Main operations:

- Login with email and password
- Logout
- Password reset / change

FR-01 – User Login

The system shall allow registered users to authenticate using their email and password.

FR-02 – Session Token

The system shall issue a JWT authentication token upon successful login, used to authorize subsequent requests.

FR-03 – Password Change

The system shall allow authenticated users to change their password.

5.2 User Management

Purpose: Manage system accounts and their roles.

Main users: SUPER_ADMIN, HR

Main operations:

- Create user accounts
- Assign roles
- Activate / deactivate accounts
- View user list

FR-04 – Role-Based Access

The system shall restrict access to system resources according to the authenticated user's role.

FR-05 – Account Status Control

The system shall allow authorized users to activate or deactivate a user account.

5.3 Employee Management

Purpose: Maintain employee HR profiles.

Main users: SUPER_ADMIN, HR

Main operations:

- Create employee profile

- Edit employee information
- Assign department, position, and manager
- View employee details
- Update employee status

FR-06 – Create Employee

The system shall allow authorized users to create an employee profile linked to a user account.

FR-07 – Update Employee

The system shall allow authorized users to update employee information, including department, position, manager, and status.

5.4 Departments

Purpose: Organize employees into departments.

Main users: SUPER_ADMIN, HR

Main operations:

- Create department
- Edit department
- Assign department manager
- View department list

FR-08 – Manage Departments

The system shall allow authorized users to create, edit, and view departments.

5.5 Positions

Purpose: Define job positions within departments.

Main users: SUPER_ADMIN, HR

Main operations:

- Create position
- Edit position
- Associate position with department

FR-09 – Manage Positions

The system shall allow authorized users to create, edit, and view positions associated with a department.

5.6 Attendance

Purpose: Track employee check-in and check-out activity.

Main users: Employee, Manager, HR

Main operations:

- Check in
- Check out
- View attendance records

- Review team attendance

FR-10 – Check-In / Check-Out

The system shall allow employees to record check-in and check-out times.

FR-11 – Attendance Status

The system shall determine attendance status (such as on time, late, or remote) based on recorded check-in information.

FR-12 – Attendance Review

The system shall allow managers and HR to view attendance records for employees under their responsibility.

5.7 Leaves

Purpose: Manage employee leave requests and approvals.

Main users: Employee, Manager, HR

Main operations:

- Submit leave request
- Select leave type and dates
- Approve or reject request
- View leave history

FR-13 – Submit Leave Request

The system shall allow employees to submit a leave request specifying a leave type and date range.

FR-14 – Leave Approval Workflow

The system shall allow authorized managers or HR to approve or reject a leave request, with a required reason for rejection.

FR-15 – Leave Notification

The system shall notify the employee when their leave request status changes.

5.8 Tasks

Purpose: Assign and track work items for employees.

Main users: Manager, Employee

Main operations:

- Create task
- Assign task to employee
- Set priority and due date
- Update task status

FR-16 – Create and Assign Task

The system shall allow managers to create tasks and assign them to employees on their team.

FR-17 – Task Status Tracking

The system shall allow the assigned employee to update the status of a task through its lifecycle (for example TODO, IN_PROGRESS, COMPLETED).

5.9 Payroll

Purpose: Calculate and manage employee salary payments.

Main users: Accountant, Employee

Main operations:

- Create payroll record
- Enter allowances, bonuses, deductions
- Calculate net salary
- View payroll status

FR-18 – Create Payroll Record

The system shall allow accountants to create a payroll record for an employee for a given period, including salary, allowances, bonuses, overtime, deductions, and tax.

FR-19 – Net Salary Calculation

The system shall calculate the net salary based on the entered payroll components.

FR-20 – View Own Payroll

The system shall allow employees to view their own payroll records.

5.10 Documents

Purpose: Store and manage employee-related documents.

Main users: Employee, HR

Main operations:

- Upload document
- Associate document with employee
- Preview document
- View document list

FR-21 – Upload Document

The system shall allow authorized users to upload documents associated with an employee profile.

FR-22 – Document Metadata

The system shall store metadata for each document, including name, type, size, and the uploading user.

5.11 Evaluations

Purpose: Record and track employee performance evaluations.

Main users: Manager

Main operations:

- Select employee
- Enter evaluation scores and comments
- Calculate overall score
- View evaluation history

FR-23 – Create Evaluation

The system shall allow authorized evaluators to create a performance evaluation for an employee, including scores and comments.

FR-24 – View Own Evaluations

The system shall allow employees to view their own evaluation history.

5.12 Notifications

Purpose: Inform users of relevant system events.

Main users: All users

Main operations:

- Receive notifications for leaves, tasks, payroll, documents, and evaluations
- View notification list
- Mark notifications as read

FR-25 – Generate Notifications

The system shall generate a notification when a relevant event occurs, such as a leave status change, a new task assignment, or a new payroll record.

5.13 Dashboards

Purpose: Provide role-specific summaries of key information.

Main users: All users

Main operations:

- Display role-relevant statistics and shortcuts

FR-26 – Role-Based Dashboard

The system shall display a dashboard appropriate to the authenticated user's role, summarizing the information most relevant to that role.

5.14 Profile & Settings

Purpose: Allow users to manage their personal account information.

Main users: All users

Main operations:

- View profile
- Edit profile
- Change password
- Manage basic account settings

FR-27 – Manage Own Profile

The system shall allow authenticated users to view and update their own profile information.

6. Non-Functional Requirements

In addition to functional behavior, SmartHR must satisfy a set of quality attributes that affect how the system performs, scales, and is maintained over time.

Security

- Password hashing
- JWT authentication
- Role-based authorization
- Protected API endpoints
- Input validation
- Controlled file uploads

Performance

- Efficient database queries
- Pagination for large datasets
- Appropriate indexing
- Optimized API communication

Scalability

- The architecture should allow future modules and features to be added without major redesign.

Maintainability

- Use of modular architecture and clear separation of responsibilities between layers.

Usability

- The interface should be simple, clean, responsive, and easy to understand.

Reliability

- The system should properly handle errors and unexpected operations without compromising data integrity.

Compatibility

- The web application should work on modern browsers and different screen sizes.

7. System Architecture

SmartHR follows a conceptual three-layer architecture: **Frontend** → **Backend API** → **Database**. Each layer has a distinct responsibility, and communication between layers happens through well-defined interfaces.

Frontend Layer

Built with: React, TypeScript (where applicable), Tailwind CSS, React Router, reusable components, and API service modules.

Responsibilities:

- User interface
- Navigation

- Forms
- Validation
- UI state
- API communication
- Authentication state
- Role-based UI rendering

Backend Layer

Built with: Node.js, Express.js, JWT, and Mongoose.

Responsibilities:

- REST APIs
- Authentication
- Authorization
- Business logic
- Validation
- Database operations
- Error handling
- File handling

Database Layer

Built with: MongoDB and Mongoose.

Responsibilities — storing:

- Users
- Employees
- HR records
- Attendance
- Leaves
- Tasks
- Payroll
- Documents metadata
- Evaluations
- Notifications

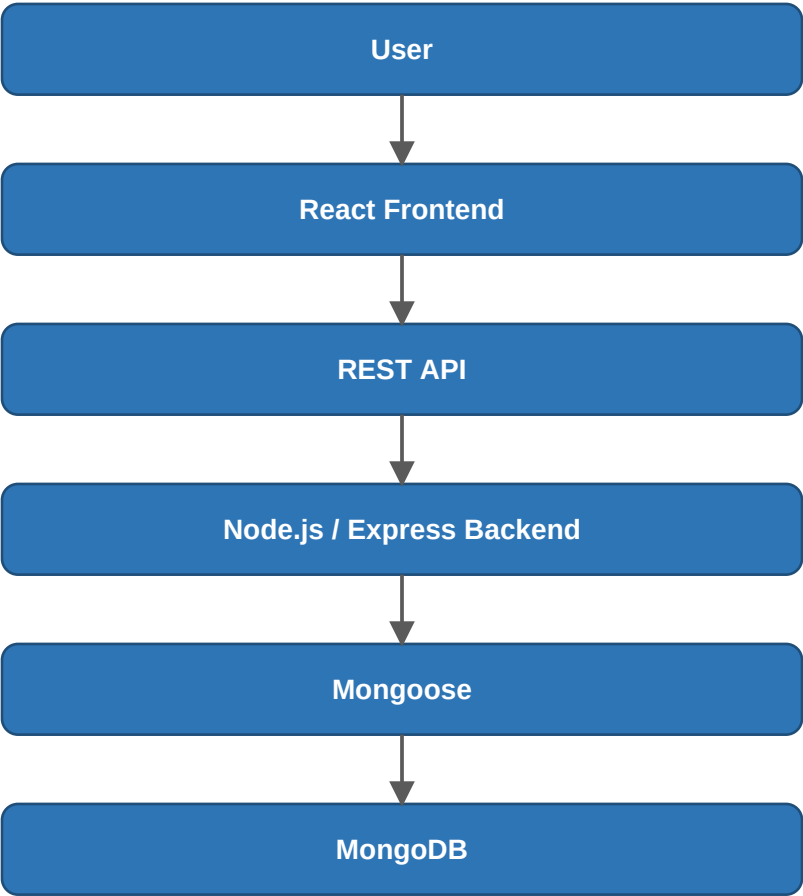


Figure 7.1 — High-level system architecture

8. Technology Stack

The table below summarizes the core technologies used to build SmartHR.

Technology	Purpose
React	Frontend UI
Node.js	Backend runtime
Express.js	REST API framework
MongoDB	Database
Mongoose	MongoDB ODM
JWT	Authentication
bcrypt	Password hashing
Tailwind CSS	UI styling
Git	Version control

Technology	Purpose
GitHub	Source code collaboration

9. Database Overview

SmartHR organizes its data into the following main collections. Each collection represents a distinct concept within the HR domain. The detailed database schema, including every field, is documented separately in the Backend Documentation.

users

Stores account identity and authentication data.

Purpose: manage login credentials, roles, and account status. Related to: employees (one user can be linked to one employee profile), and notifications.

employees

Stores HR/business profile information for each staff member.

Purpose: hold employment details such as employee code, department, position, manager, salary information, and employment status. Related to: users, departments, positions, attendance, leaves, tasks, payroll, documents, and evaluations.

departments

Stores organizational department information.

Purpose: group employees and positions under a department. Related to: employees and positions.

positions

Stores job position information.

Purpose: define roles/titles within a department. Related to: departments and employees.

attendances

Stores daily attendance records.

Purpose: record check-in/check-out times, status, and working hours. Related to: employees.

leaves

Stores employee leave requests.

Purpose: track leave type, date range, status, and approval information. Related to: employees.

tasks

Stores assigned work items.

Purpose: track task details, priority, status, and due dates. Related to: employees (assignee and, conceptually, assigner).

payrolls

Stores monthly/periodic payroll records.

Purpose: hold salary components and computed net salary for a given period. Related to: employees.

documents

Stores metadata about employee documents.

Purpose: track uploaded files associated with an employee profile. Related to: employees and users (uploader).

evaluations

Stores employee performance evaluations.

Purpose: hold evaluator information, scores, comments, and overall score. Related to: employees.

notifications

Stores system notifications for users.

Purpose: inform users about relevant events across modules. Related to: users.

10. Main Relationships

This section explains the important relationships between the main entities in the system.

User → Employee

A User represents the authentication/account identity, while an Employee represents the HR/business profile. An employee is linked to a user through **userId**. These are kept separate because a User handles authentication and authorization, while an Employee handles HR information.

Department → Employees

A department can contain multiple employees.

Department → Positions

A department can have multiple positions.

Employee → Manager

An employee may have a manager, who is themselves another employee.

Employee → Attendance

An employee can have multiple attendance records.

Employee → Leaves

An employee can submit multiple leave requests.

Employee → Tasks

An employee can have multiple assigned tasks.

Employee → Payroll

An employee can have multiple payroll records for different months.

Employee → Documents

An employee can have multiple documents.

Employee → Evaluations

An employee can have multiple evaluations.

User → Notifications

A user can receive multiple notifications.

11. Authentication & Authorization Flow

SmartHR relies on JWT-based authentication combined with role-based authorization. The flow below explains, step by step, how a user gains access to the system and how requests are validated.

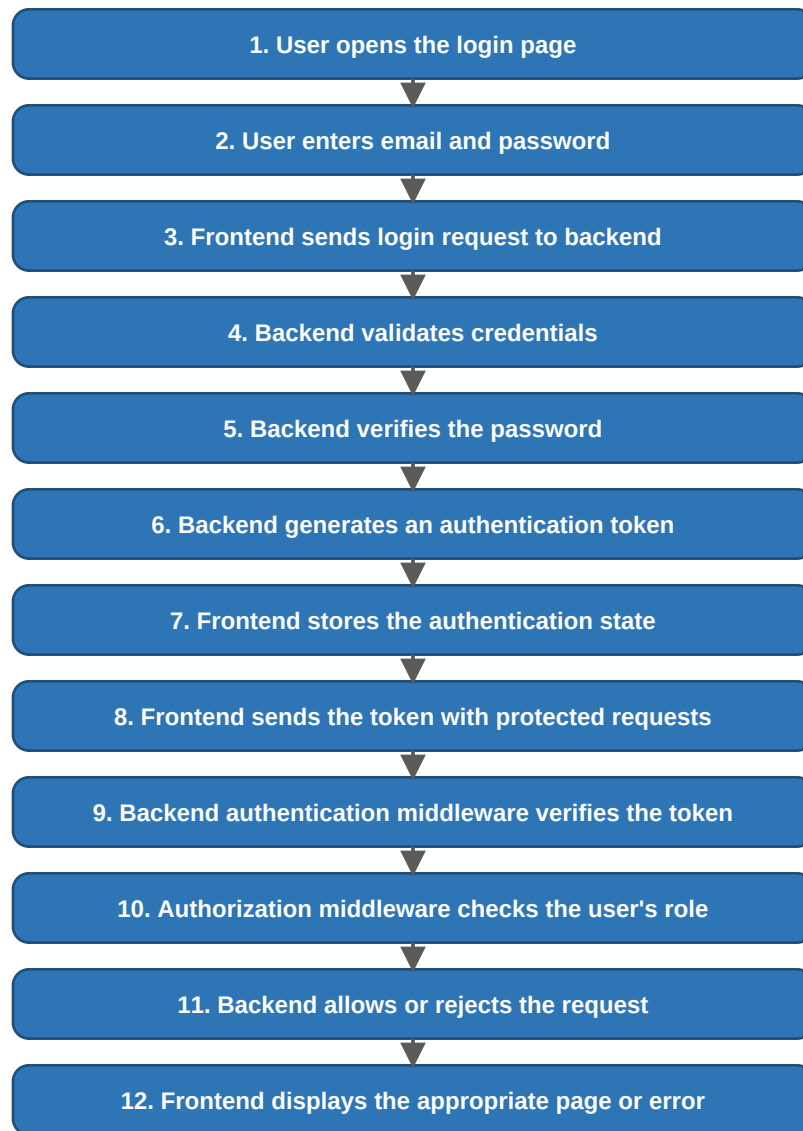


Figure 11.1 — Authentication and authorization flow

Authentication = Who are you? The process of verifying the identity of the user attempting to access the system.

Authorization = What are you allowed to do? The process of determining whether an authenticated user's role permits a specific action or resource.

12. Core System Workflows

This section explains the most important workflows in SmartHR at a conceptual level.

12.1 Employee Creation Workflow

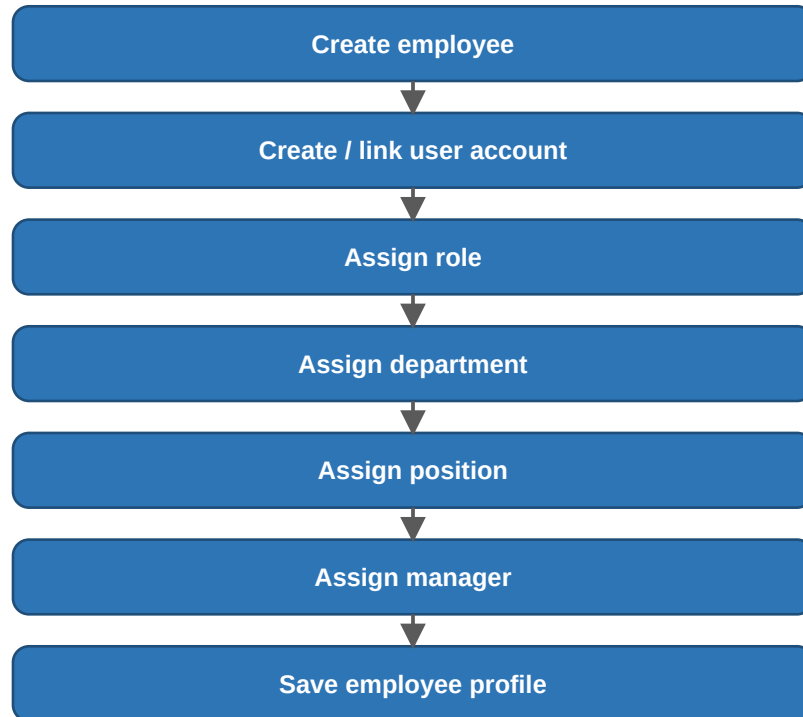


Figure 12.1 — Employee creation workflow (performed by HR / Admin)

12.2 Leave Request Workflow

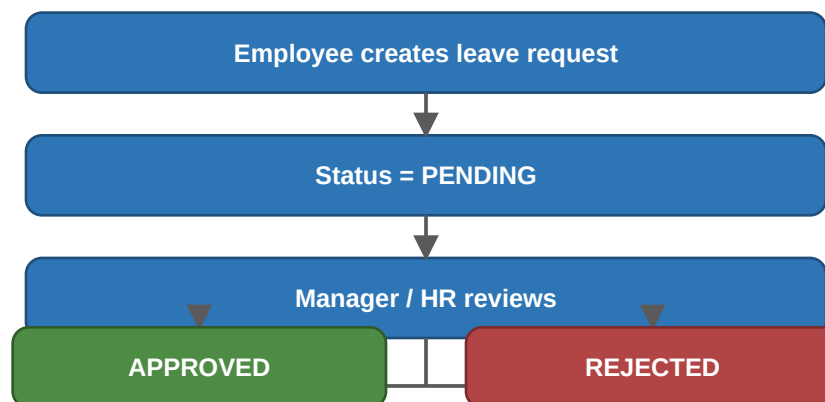


Figure 12.2 — Leave request workflow. If rejected, the system stores the rejection reason; the employee receives a notification of the outcome either way.

12.3 Task Workflow

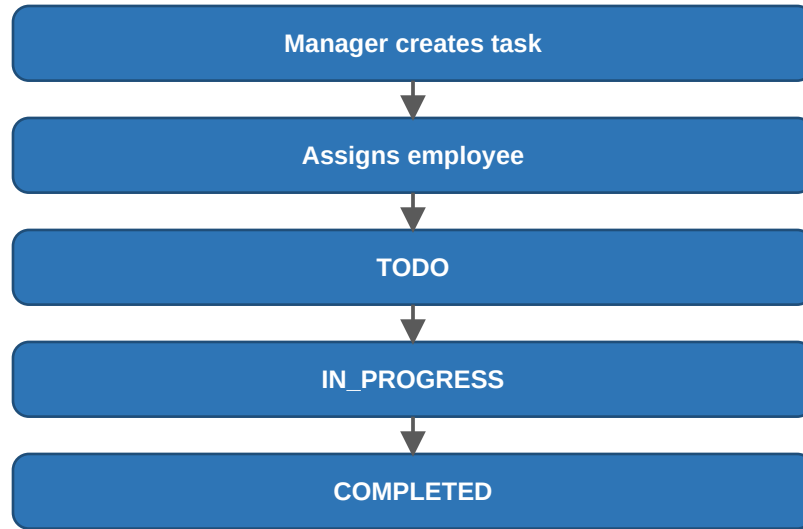


Figure 12.3 — Task workflow. Task status can be tracked throughout its lifecycle.

12.4 Attendance Workflow

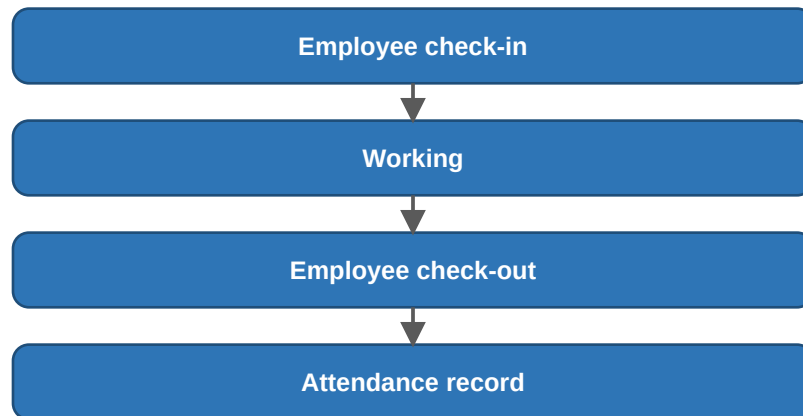


Figure 12.4 — Attendance workflow. The system can calculate working hours, late minutes, and overtime from the recorded times.

12.5 Payroll Workflow

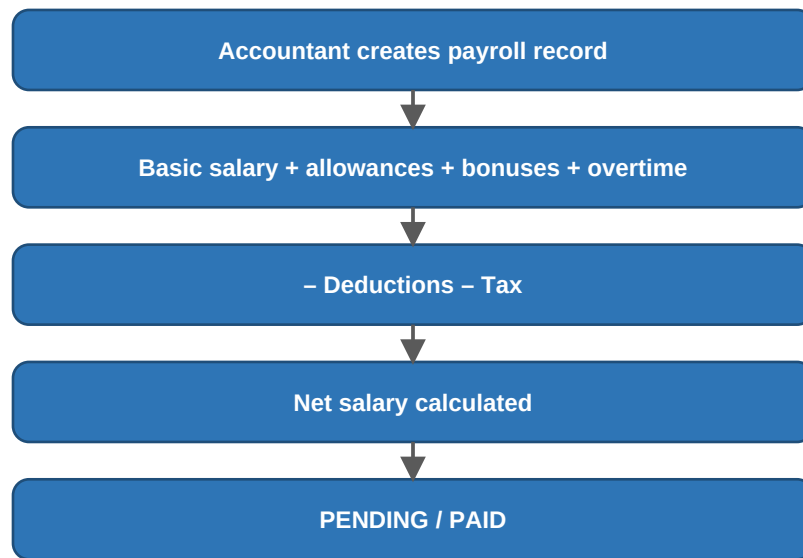


Figure 12.5 — Payroll workflow

12.6 Evaluation Workflow

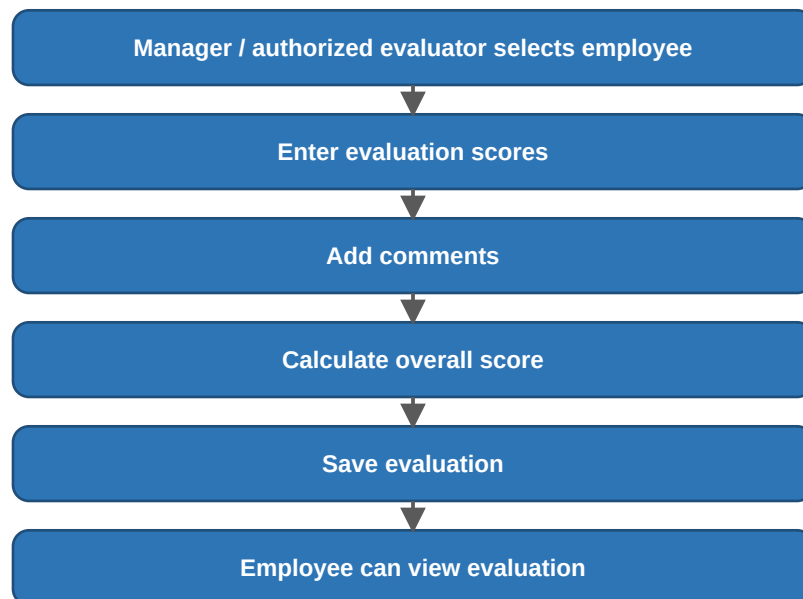


Figure 12.6 — Evaluation workflow

13. Frontend & Backend Responsibilities

The table below clarifies which responsibilities belong to the frontend, the backend, or both. Frontend restrictions improve the user experience, but **backend authorization is the real security boundary** of the system.

Responsibility	Frontend	Backend
UI rendering	✓	—
Form interaction	✓	—
Client-side validation	✓	✓
Authentication UI	✓	✓
JWT verification	—	✓
Role authorization	UI restriction	✓
Business logic	Limited	✓
Database access	—	✓
API endpoints	—	✓
Error presentation	✓	✓
File upload interface	✓	✓
File storage / processing	—	✓

14. API Communication Overview

The frontend communicates with the backend exclusively through REST APIs. The conceptual flow of a request is as follows:

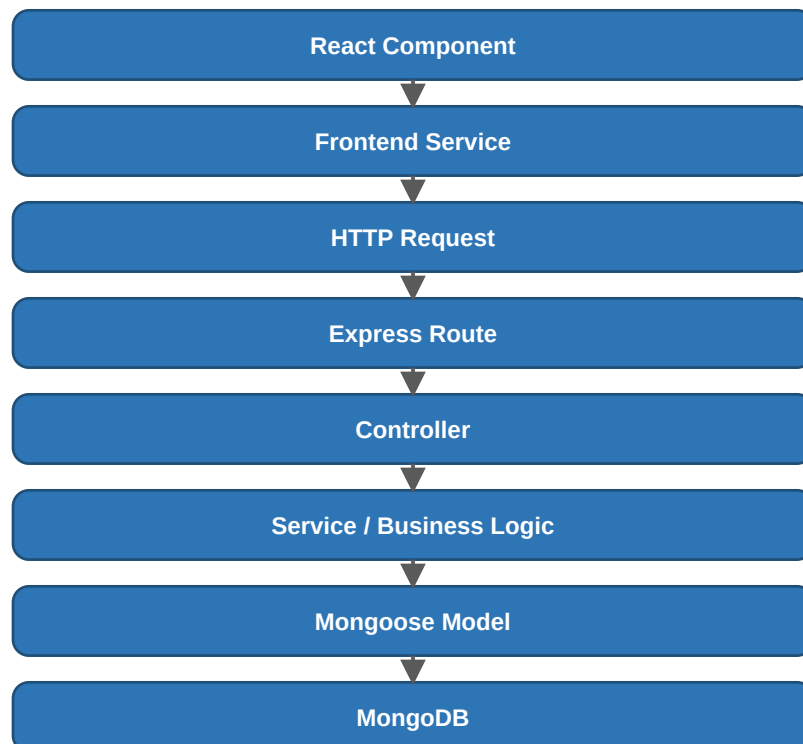


Figure 14.1 — API communication flow. Responses travel back through the same layers in the opposite direction.

Common HTTP methods used across the API:

- **GET** – retrieve data
- **POST** – create data
- **PUT / PATCH** – update data
- **DELETE** – remove data

This shared document does not enumerate every endpoint; the complete endpoint list is documented separately in the Backend Documentation.

15. UI/UX Design Principles

SmarHR follows a simple and practical, MVP-oriented design philosophy. The interface favors clarity and consistency over decoration, so that every user role can complete their tasks quickly and without confusion.

The design relies on the following elements:

- Clean interface
- Simple navigation
- Consistent layout
- Reusable components
- Clear forms
- Standard tables
- Simple cards
- Status badges
- Simple dashboards
- Clear error messages
- Loading states
- Empty states
- Responsive design

Important: the UI intentionally avoids:

- Unnecessary decorative elements
- Complex animations
- Overly complicated charts
- Excessive dashboard widgets
- Complicated interactions

Design goal: Professional + Simple + Easy to Use + Easy to Implement.

16. Main Application Screens

This section provides a high-level overview of the main screens in SmartHR, grouped by functional area. The exact UI implementation of each screen is documented separately in the Frontend Documentation.

Group	Screens
Authentication	Login, Forgot Password, Reset Password, Change Password
Dashboards	Super Admin Dashboard, HR Dashboard, Manager Dashboard, Accountant Dashboard, Employee Dashboard
Users & Employees	Users Management, User Details, Employees Management, Employee Details, Add Employee, Edit Employee
Organization	Departments, Department Details, Add/Edit Department, Positions, Add/Edit Position
Attendance	Attendance Dashboard, Attendance Management, My Attendance
Leaves	My Leaves, Request Leave, Leave Requests, Leave Details
Tasks	Tasks Dashboard, My Tasks, Team Tasks, Create Task, Task Details, Edit Task
Payroll	Payroll Dashboard, Payroll Management, Create Payroll, Payroll Details, My Payroll
Documents	Documents, Upload Document, Document Details / Preview
Evaluations	Evaluations, Evaluation Details, Create Evaluation, Edit Evaluation
Notifications	Notification Center, Notification Dropdown
Profile & Settings	My Profile, Edit Profile, Settings

17. Error Handling & System States

SmartHR defines a consistent set of system states that appear across modules, so that users always receive clear feedback about what is happening.

- Loading
- Empty State
- Error State
- Success State
- Form Validation
- Delete Confirmation
- Toast Notifications
- Search No Results
- Permission Denied / 403
- Unauthorized / 401

These states are important because they give users continuous, predictable feedback: loading and empty states set correct expectations, error and validation states help users correct problems, and permission-related states clearly communicate access boundaries. Together, they contribute directly to the system's usability and perceived reliability.

18. Security Overview

This section summarizes the major security considerations built into SmarHR's design.

Authentication

JWT-based authentication is used to verify user identity.

Password Security

Passwords are never stored as plain text; secure password hashing such as bcrypt is used.

Authorization

Every protected backend resource verifies the user's role and permissions before granting access.

Input Validation

Incoming data is validated before being processed by the system.

File Upload Security

Uploaded files are restricted according to allowed types and size limits.

API Security

Sensitive endpoints are protected, and unauthorized requests are rejected.

Database Security

Database access is controlled and incoming data is validated before being persisted.

Note: this document does not claim advanced security mechanisms beyond those described above unless they are actually implemented.

19. File Management

Employees can have documents associated with their HR profile — for example contracts, certificates, or identification documents. The system stores metadata describing each document rather than the raw file content itself.

Stored document metadata includes:

- Document name
- Type
- URL
- File size
- MIME type
- Uploading user
- Employee
- Creation date

The actual file storage can be handled by an external file storage service, such as Cloudinary, if that is the chosen implementation.

It is important to clearly distinguish between:

- **File Storage** — where the raw file bytes physically reside (e.g. an external storage service).
- **Database Metadata** — the descriptive record stored in MongoDB that points to the file and describes it.

20. Project Structure Overview

The high-level project structure separates the frontend, backend, and documentation into distinct top-level folders:

```
SmarHR/  
|  
+-- frontend/  
|  
+-- backend/  
|  
+-- documentation/  
| +-- project-documentation  
| +-- frontend-documentation  
| +-- backend-documentation  
|  
+-- README.md
```

frontend/ — contains the React application (UI, components, pages, services).

backend/ — contains the Node.js / Express application (routes, controllers, models, middleware).

documentation/ — contains the three project documents: this shared project documentation, the Frontend Documentation, and the Backend Documentation.

README.md — provides a general entry point describing the project and how to run it.

Detailed source-code folder structures are not included here, as they belong to the Frontend and Backend documentation respectively.

21. Development & Collaboration

The team collaborates using standard software engineering practices:

- Git
- GitHub
- Feature branches
- Pull requests
- Code reviews
- Issue tracking
- Separate frontend/backend responsibilities
- Shared project documentation

Example branching workflow:

```
main
|
+-- feature/auth
+-- feature/employees
+-- feature/attendance
+-- feature/payroll
+-- feature/tasks
```

Developers are expected to avoid directly modifying **main** when working on a feature; instead, changes are made on a dedicated feature branch and merged through a reviewed pull request.

22. Testing Overview

This section provides a high-level testing strategy for SmarHR. Specific testing frameworks are not named unless they are actually being used by the team.

Frontend Testing

- Form validation
- Component behavior
- Navigation
- Authentication UI
- Role-based UI
- API integration

Backend Testing

- Authentication
- Authorization
- API endpoints
- Validation
- Business logic
- Error handling

Integration Testing

Tests the communication between: Frontend → Backend → Database, ensuring that data flows correctly across all three layers.

Manual Testing

Verifies complete user workflows for every role, confirming that each role can perform its intended operations end-to-end.

23. Future Enhancements

The items below are possible future improvements to SmarHR. They are **not part of the current MVP** and are listed here only as ideas for potential future development.

- Advanced HR analytics
- Advanced reporting
- Email notifications
- Real-time notifications
- Calendar integration
- Employee self-service improvements
- Attendance device integration
- Advanced payroll calculations
- Performance history
- Audit logs
- Multi-company support
- Mobile application
- Advanced search and filtering

24. Documentation Structure

The complete SmartHR project documentation is divided into three documents. This shared document provides the **big picture**, while the Frontend and Backend documents provide the **implementation details**.

1. Project Documentation (this document)

- Project overview
- Objectives
- Scope
- Roles
- Requirements
- Architecture
- Workflows
- Technology stack
- Database overview
- Security overview
- UI/UX principles

2. Frontend Documentation

- React architecture
- Folder structure
- Components
- Pages
- Routing
- State management
- Forms
- Validation

- API integration
- Authentication state
- UI implementation
- Frontend services

3. Backend Documentation

- Node.js architecture
- Express structure
- Folder structure
- Models
- Database schemas
- Controllers
- Services
- Routes
- Middleware
- Validators
- Authentication
- Authorization
- API endpoints
- Error handling
- File upload
- Database relationships

25. Glossary

Term	Definition
API	Application Programming Interface — a set of rules that lets software components communicate with each other.
REST API	An architectural style for web APIs that uses standard HTTP methods to operate on resources.
JWT	JSON Web Token — a compact, signed token format used to represent an authenticated user's identity.
Authentication	The process of verifying who a user is.
Authorization	The process of determining what an authenticated user is allowed to do.
Role-Based Access Control	A security model that grants permissions based on a user's assigned role.
CRUD	Create, Read, Update, Delete — the four basic operations performed on data.
MongoDB	A NoSQL, document-oriented database used to store SmarHR's data.
Mongoose	An Object Data Modeling (ODM) library that provides schema-based modeling for MongoDB in Node.js.
Middleware	A function that runs during the request/response cycle, often used for tasks such as authentication or validation.
Frontend	The client-side part of the application that users interact with directly.
Backend	The server-side part of the application that handles business logic and data access.
HTTP	HyperText Transfer Protocol — the protocol used for communication between client and server on the web.
Endpoint	A specific URL path exposed by the backend API for a particular operation.
Collection	A group of documents in MongoDB, similar to a table in a relational database.
Document	A single record stored in a MongoDB collection, similar to a row in a relational table.
ObjectId	A unique identifier automatically generated by MongoDB for each document.

26. Conclusion

SmarHR provides a centralized platform for managing employees and HR operations, bringing together employee records, attendance, leave requests, tasks, payroll, documents, evaluations, and notifications into a single, consistent system.

The architecture separates frontend, backend, and database responsibilities into clear layers, and uses role-based access control to provide different capabilities for different users — from the SUPER_ADMIN and HR staff who manage the organization, to managers who support their teams, accountants who handle payroll, and employees who manage their own information.

The project is designed to be:

- Simple

- Secure
- Maintainable
- Scalable
- User-friendly
- Suitable for future expansion

Together with the Frontend Documentation and Backend Documentation, this shared document is intended to give any team member, evaluator, or new developer a clear understanding of what SmartHR is, who it serves, and how its parts fit together before diving into implementation-specific details.